Accuracy, Stability, and Repeated-Run Reliability of Large Language Models on Deterministic Programming Tasks

Anonymous ACL submission

Abstract

Run-level pass rate overstates retry-free coverage by up to 17.8 percentage points—and the gap is largest precisely for mid-performing systems. We investigate this accuracy–stability relationship in large language model (LLM) evaluation for deterministic text-conditioned generation, using programming tasks as a concrete testbed. Standard code-generation benchmarks emphasize single-run accuracy or eventual success under repeated sampling, but many deployment settings also require *stability*: consistent outcomes across repeated invocations under the same task description. We present a repeated-run evaluation protocol with metrics for run-level accuracy, retry-free coverage, and per-problem variability. On a recency-based benchmark of 100 LeetCode-style problems, we evaluate 16 models from five provider families under two prompt templates with five repeated runs per problem, yielding 16,000 evaluation instances. Although run-level pass rate and perfect stability rate are strongly correlated ($r = 0.985$), pass rate consistently exceeds retry-free coverage—a gap that reaches 17.8 percentage points and reverses model rankings even among closely matched systems. Prompt effects are model-dependent rather than uniformly beneficial. These results suggest that repeated-run stability analysis is a necessary complement to conventional accuracy reporting for deterministic text-conditioned generation tasks.

1 Introduction

Large language models are increasingly used for text-conditioned tasks with deterministic specifications: given a fixed textual input and a fixed verifier, correctness is binary. Program synthesis and code assistance are especially important instances of this broader setting, because they pair natural-language problem statements with executable test suites. Conventional benchmarks typically report single-run performance, or summarize repeated sampling through eventual-success metrics such as pass@k (Chen et al., 2021; Austin et al., 2021). Recent work has shown that repeated sampling can be scaled as a form of inference-time compute (Brown et al., 2024), yet the *stability* of outcomes across such repeated invocations—whether a model consistently succeeds or fails on the same task—has received less systematic attention. In practice, repeated calls under the same task description can yield different outputs and different pass/fail outcomes, even when the downstream workflow expects reproducible behavior (Atil et al., 2024; Alvarado Gonzalez et al., 2025).

This paper studies that repeated-invocation variability directly. Rather than treating repeated sampling only as an opportunity to recover one success among many attempts, we investigate the relationship between correctness and stability under fixed conditions. Our focus is therefore complementary to pass@k (Chen et al., 2021): whereas pass@k asks whether a model can produce *at least one* correct solution in k attempts—a capability measure—we ask how reliably it succeeds on *every* invocation, a deployment reliability measure. These two questions target different downstream concerns and can yield divergent rankings, particularly for mid-accuracy systems.

The paper is organized around three empirical questions:

1. How large is the gap between run-level success and retry-free problem coverage under repeated evaluation?

2. Which aggregate summaries best expose problem-level variability that single-run reporting hides?

3. As an exploratory analysis: do prompt choices or reasoning-specialized architectures systematically shift repeated-run stability?

We focus on deterministic tasks because they represent a broader and practically important class of NLP system behavior: the model is conditioned on text, but the output is consumed by a verifier that expects reproducible correctness. Code-generation systems are a particularly stringent instance of this pattern, alongside tasks such as text-to-SQL (Gao et al., 2024), structured semantic parsing, tool-call argument generation, and formal text transformations. In such settings, the accuracy–stability gap has direct practical consequences: a model with 60% run-level pass rate but only 47% perfect stability solves many individual attempts, yet still leaves a substantial fraction of tasks in a retry-required regime. Our aim is therefore not merely to rank models by mean performance, but to characterize how much of their observed accuracy translates into reliable repeated-use behavior.

2 Related Work

Evaluations of LLMs for code generation are commonly framed around functional correctness and pass@k under automated testing (Chen et al., 2021; Austin et al., 2021). Foundational benchmarks such as HumanEval, MBPP, and APPS emphasize whether a model can eventually produce at least one correct program under repeated sampling (Chen et al., 2021; Austin et al., 2021; Hendrycks et al., 2021), while competition-oriented settings such as AlphaCode focus on large-scale sampling and selection for difficult programming tasks (Li et al., 2022). More recent benchmarks, including LiveCodeBench and SWE-bench, have further highlighted contamination, temporal validity, and realistic task design as central evaluation concerns (Jain et al., 2024; Jimenez et al., 2024). Our work is complementary to these efforts: rather than introducing a new benchmark family, we study what repeated sampling reveals about the relationship between correctness and stability on deterministic code tasks.

Related prompting work, including zero-shot reasoning, self-consistency, and adaptive-consistency sampling, also relies on multiple samples, but typically treats them as a mechanism for improving downstream task performance rather than as an object of measurement in its own right (Kojima et al., 2022; Wang et al., 2022; Aggarwal et al., 2023). A complementary body of work has directly studied LLM output non-determinism and found that performance can vary substantially across repeated runs even under nominally identical conditions (Atil et al., 2024; Alvarado Gonzalez et al., 2025). Prompt sensitivity research has further shown that small changes to instruction phrasing can shift model accuracy significantly (Zhuo et al., 2024), a concern our study controls for by holding prompt templates fixed within each condition.

Our work does not claim novelty at the level of inventing fundamentally new mathematical quantities: both retry-free coverage and per-problem variability can be expressed using standard repeated-sampling summaries. Specifically, PSR is mathematically equivalent to pass@k evaluated at $k = R$ with the threshold set to "all successes"; however, the two metrics answer *different questions*. Pass@k estimates $P(\text{at least one success in } k \text{ draws})$—a *capability* measure. PSR estimates $P(\text{all } k \text{ draws succeed})$—a *deployment reliability* measure. These two quantities diverge most sharply for mid-accuracy systems, and the gap between them has direct practical consequences. The novelty lies in treating these summaries as first-class evaluation targets for deterministic code tasks, placing them alongside AV to characterize the accuracy–stability gap, and using them to study the correctness–stability relationship directly.

Recent work has also emphasized that code-generation evaluation is sensitive to benchmark construction, execution methodology, and reporting choices (Liu et al., 2024; Du et al., 2024). Our framing is most aligned with this robustness-oriented perspective, but shifts the emphasis from benchmark validity alone to repeated-invocation reliability under a fixed judge. Instead of asking only whether a benchmark is sound or whether a model can eventually solve a task, we ask how often correctness is reproducible at the problem level when the task, prompt family, and decoding regime are held fixed. Although we instantiate this methodology on coding problems, the same evaluation logic applies more broadly to deterministic text-conditioned generation tasks with verifiable outputs.

More broadly, work on calibration and predictive reliability (Guo et al., 2017) and on performance–efficiency trade-offs in model selection (Zhang et al., 2026) motivates looking beyond average accuracy when systems are used in settings where consistency matters. We adapt that intuition to deterministic program evaluation by measuring not only run-level success, but also retry-free task cov-

erage and variability across repeated runs. This perspective is deliberately narrower than end-to-end agent evaluation: it isolates repeated-run behavior for a single-turn coding setting, while real deployments may involve additional orchestration, tool use, and runtime dependencies whose reliability and security properties must be analyzed separately.

3 Experimental Setup

3.1 Problem Dataset

We evaluate on a curated dataset of deterministic programming problems, denoted $\mathcal{D} = \{(x_i, T_i)\}_{i=1}^{N}$, where x_i is the natural-language specification (including I/O format and constraints) and T_i is a deterministic test harness. Each problem is graded by executing the model-produced program against T_i, producing a binary correctness indicator. These tasks are instantiated as LeetCode problems with platform-provided acceptance criteria, and all generations are evaluated as Python solutions against Python-based submission templates.

Problem selection. To reduce the risk of benchmark contamination from model training data, we select the $N = 100$ most recently published problems available on the platform at collection time, sampling the newest problems first. Problems requiring paid-only access are excluded. No further manual curation is applied; the final benchmark is entirely determined by recency rank. This construction does not guarantee that evaluated models have never encountered related problem statements during training, but it substantially reduces the probability of direct memorization compared to historically popular problems. We note, however, that recency alone does not guarantee a contamination-free evaluation: recently published contest problems are often discussed publicly online—including on forums, social media, and editorial sites—within days of release. Post-release discussion means that problem statements, constraints, and accepted solutions may appear in model training corpora even for recently added problems, and we do not perform contamination-detection analysis (e.g., via prompt perturbation, n-gram overlap, or membership inference (Deng et al., 2023)). This limitation is discussed further in the Limitations section.

The dataset spans three difficulty tiers: 20 Easy, 50 Medium, and 30 Hard problems, covering a broad range of algorithmic topics including Array, Dynamic Programming, String, Graph, and Simulation. All problems satisfy the following properties:

- **Deterministic grading:** each T_i yields a deterministic accept/reject outcome.
- **Self-contained evaluation:** no network access or external dependencies are required at runtime.
- **Fixed snapshot:** all models are evaluated on the identical problem set to ensure comparability across runs.

In the evaluation framework, this benchmark is stored as a fixed local dataset snapshot rather than fetched online at run time. This design ensures that all model families are evaluated against the same problem set and ordering, and reduces the chance that later platform changes silently alter the benchmark during the experimental campaign.

3.2 Models

We evaluate 16 models across five provider families, covering a range of model scales, training objectives, and architectural choices, including both standard and reasoning-specialized variants. The evaluated models are listed in Table 1.

Table 1: Models evaluated in this study, grouped by provider family. † denotes reasoning-specialized models. Display names in this table are editorial labels derived from API identifier strings; they are not official product version names. Exact API identifiers and access dates are listed in Appendix B. Google models marked "(preview)" were accessed via preview-tier API endpoints.

Family	Model	Type
OpenAI	GPT-4.1	flagship
	GPT-4.1-mini	lightweight
	o4-mini	reasoning†
Anthropic	Claude Sonnet 4.5	flagship
	Claude Haiku 4.5	lightweight
Google	Gemini 3.1 Pro (preview)	flagship
	Gemini 3 Flash (preview)	efficient
	Gemini 3.1 Flash-Lite (preview)	lightweight
	Gemini 2.5 Flash	efficient
	Gemini 2.5 Flash+Think	reasoning†
Qwen	QwQ-Plus	reasoning†
	Qwen-Plus	flagship
	Qwen-Max	large
	Qwen-Turbo	lightweight
DeepSeek	DeepSeek-R1	reasoning†
	DeepSeek-V3.2	flagship

3.3 Evaluation Framework

For a model m and a fixed evaluation configuration c (prompt template, decoding parameters, toolchain), we run each problem x_i multiple times. Each run produces a candidate program which is executed against T_i.

Let $Y_{i,r}^{(m,c)} \in \{0,1\}$ denote whether run $r \in \{1,\dots,R\}$ passes all tests for problem i. We treat compilation errors, runtime errors, timeouts, and wrong answers as failures (0).

The evaluation pipeline separates generation from execution. For large-scale experiments, model outputs are first generated through provider APIs, cached locally, normalized into a shared format, and only then submitted to the deterministic judge. This design makes experiments resumable across providers and reduces the risk that transient provider-side failures are conflated with downstream execution outcomes.

3.4 Repeated-Run Methodology

We adopt a repeated-run design to estimate both mean performance and variability. For each (m, c) pair we:

1. Sample R independent generations per problem under configuration c.
2. Evaluate each generation deterministically against T_i.
3. Aggregate outcomes into run-level and problem-level metrics with uncertainty estimates.

The primary controlled intervention is the prompt template. We compare two prompt variants while holding the dataset, model, decoding parameters, and execution environment fixed. All models are evaluated with temperature 0.3, top-$p = 0.9$, and $R = 5$ repeated runs per problem per prompt configuration. The default maximum output budget is 4,096 tokens; reasoning-specialized models (o4-mini, Gemini 2.5 Flash+Think) use larger budgets to accommodate extended chain-of-thought output (see Appendix B for per-model details). The o-series API (o4-mini) does not support explicit temperature or top-p settings; decoding parameters are omitted for that model (implications discussed in the Limitations section). We use a non-zero temperature because the goal of the study is to characterize repeated-run variability under realistic stochastic decoding rather than to collapse outputs toward near-deterministic behavior at $T = 0$.

The two prompt templates—DETAILED and MINIMAL—are held fixed across all models to enable a controlled comparison of prompt sensitivity.

Prompt templates. Both prompts instruct the model to return plain Python source code only and to follow the provided LeetCode method signature exactly. The DETAILED prompt uses a longer structured template emphasizing careful constraint reading, edge-case coverage, submission readiness, and accepted-style performance. The MINIMAL prompt conveys the same task more compactly, with lighter instruction scaffolding. Because both templates share the same output constraints and code template, the prompt comparison is a controlled low-contrast comparison that isolates instruction density rather than changing the task itself. Accordingly, the study does not test broad prompt-engineering strategies; it evaluates a narrow prompt variation under otherwise fixed conditions.

Normalization and execution. Raw generations from different providers are normalized into a common local representation before evaluation. Code is extracted from model responses, basic syntax validity is checked, and each candidate is then submitted to the same deterministic judge. This normalization step is important because provider APIs differ in output packaging, token accounting, and batch interfaces even when the downstream task is identical.

3.5 Metrics Definition

Run-Level Pass Rate (RLPR).

$$\mathrm{RLPR}(m,c) = \frac{1}{NR}\sum_{i=1}^{N}\sum_{r=1}^{R} Y_{i,r}^{(m,c)}.$$

RLPR measures the probability that a *random invocation* succeeds.

Perfect Stability Rate (PSR).

$$\mathrm{PSR}(m,c) = \frac{1}{N}\sum_{i=1}^{N}\mathbf{1}\left[\sum_{r=1}^{R} Y_{i,r}^{(m,c)} = R\right].$$

PSR quantifies the fraction of tasks for which the system is reliably correct without retries.

Average Variance (AV).

$$\mathrm{AV}(m,c) = \frac{1}{N}\sum_{i=1}^{N}\frac{R}{R-1}\hat{p}_i^{(m,c)}\left(1-\hat{p}_i^{(m,c)}\right),$$

where $\hat{p}_i = \frac{1}{R}\sum_r Y_{i,r}$. This finite-sample correction yields the unbiased Bernoulli variance estimate for each problem under repeated sampling. AV captures average instability across tasks.

95% Confidence Intervals. For RLPR and PSR, we compute 95% Wilson score intervals. For AV, we use nonparametric bootstrap over problems.

Prompt comparison tests. Because DETAILED and MINIMAL prompting are evaluated on the same underlying problems, prompt comparisons are paired rather than independent. For PSR, we therefore compare prompt conditions at the problem level using an exact McNemar test on the binary indicator of whether all $R = 5$ runs for a problem pass under each prompt condition. We use these tests as a conservative check on whether observed prompt differences are larger than expected from the finite benchmark size alone.

Scope of inference. Unless otherwise stated, reported statistics are descriptive summaries of the observed benchmark and evaluation protocol rather than claims of universal model behavior. In particular, prompt comparisons should be interpreted as controlled comparisons between the two specific templates used in this study, not as claims about prompt engineering in general.

4 Results

4.1 Overall Accuracy and Stability

Table 2 reports RLPR, PSR, and AV for all 32 model/prompt configurations. The top performers under RLPR are o4-mini (86.2% D / 85.6% M), Gemini 3.1 Pro (86.2% D / 85.8% M), and DeepSeek-R1 (84.2% D / 84.4% M). More broadly, the table shows that high run-level performance and high retry-free coverage often co-occur, but not in a one-to-one way: models with similar RLPR can still differ materially in PSR and AV.

Figure 1 plots RLPR against PSR for all 32 configurations. All points fall below the diagonal, confirming that RLPR consistently overstates production reliability. Across all configurations, RLPR and PSR are strongly correlated ($r = 0.985$, $p < 0.001$), which is directionally expected because both metrics are derived from the same underlying per-problem success structure. The informative result is therefore not the existence of a high correlation itself, but the persistent and non-uniform gap between run-level success and retry-free coverage.

The gap $\Delta = \text{RLPR} - \text{PSR}$ is non-uniform and reaches its maximum in the mid-accuracy tier. Gemini 2.5 Flash has the largest gap (17.8% under DETAILED), followed by Gemini 3.1 Flash-Lite (16.2% D) and Gemini 2.5 Flash+Think (17.4% M). At the low end, Qwen-Turbo has a gap of only 2.4%—not because it is uniformly reliable, but because its outcomes are highly polarized: many problems are either always failed or always passed, leaving less room for intermediate trial-to-trial variability. This pattern is reflected in AV: Qwen-Turbo has the lowest AV (0.0088), while Gemini 2.5 Flash has the highest (0.0704). Together, PSR and AV help distinguish between systems that are consistently strong, consistently weak, and unstable in the middle.

While it is mathematically expected that RLPR $\geq$ PSR for any non-trivial Bernoulli process, and that the two metrics are highly correlated overall, the *non-uniformity* of Δ across models is the substantive empirical finding. This non-uniformity is itself structurally expected: Bernoulli variance is maximized at $p \approx 0.5$, so mid-performing models—whose per-problem success probabilities cluster in the intermediate range—will exhibit the largest gaps by construction. The practical contribution is that this gap is observable, model-specific, and meaningfully large (up to 17.8 percentage points), meaning that RLPR rankings alone cannot reveal how much of a model's accuracy translates into retry-free reliability.

Knowing the theoretical direction does not eliminate the need to measure the magnitude. The empirically determined quantities—how large Δ actually is (up to 17.8 pp) and which specific models fall in the mid-accuracy danger zone—cannot be derived from first principles, since per-problem success probabilities are not known a priori. The contribution is therefore best understood as empirical calibration of a theoretically expected pattern at scale.

4.2 Problem-Level Stability Analysis

Figure 2 presents a cross-model stability overview. Each row corresponds to one of the 16 models, ordered by RLPR under DETAILED prompting (highest at top). Each column corresponds to one of the 100 problems, grouped by difficulty tier (Easy, Medium, Hard) and sorted within each tier by decreasing mean pass rate across all models. Cell color encodes the empirical pass probability for that model–problem pair, averaged over $R = 5$

Table 2: Main results across all 16 models and two prompt configurations (100 problems, $R = 5$ runs, 16,000 total evaluation instances). RLPR and PSR are in %; AV is shown $\times 10^{-2}$ (i.e., multiply by 0.01 for the raw value). All entries are point estimates; 95% CIs (Wilson score for RLPR/PSR; bootstrap for AV) available from authors on request. † reasoning-specialized model. $*$ output budget differs from the 4,096-token default: Gemini 2.5 Flash+Think used 24,576 tokens (8,192 thinking); o4-mini used 16,384 tokens. ‡ o4-mini decoding is not user-controllable; its PSR/AV may reflect reduced stochasticity rather than task reliability—see the Limitations section.

Family	Model	RLPR (%)		PSR (%)		AV ($\times 10^{-2}$)	
		D	M	D	M	D	M
OpenAI	GPT-4.1	59.6	59.2	47.5	49.0	4.36	4.08
	GPT-4.1-mini	58.4	59.2	50.0	47.0	3.28	4.48
	o4-mini†‡	86.2	85.6	75.0	69.0	3.36	5.20
Anthropic	Claude Sonnet 4.5	52.8	51.4	43.0	47.0	3.36	1.68
	Claude Haiku 4.5	41.2	40.8	35.0	34.0	2.40	2.00
Google	Gemini 3.1 Pro (preview)	**86.2**	85.8	**75.0**	74.0	3.36	3.92
	Gemini 3 Flash (preview)	80.8	**83.2**	67.0	70.0	5.28	4.40
	Gemini 3.1 Flash-Lite (preview)	69.2	63.6	53.0	50.0	5.28	5.44
	Gemini 2.5 Flash*	53.8	48.2	36.0	33.0	7.04	5.84
	Gemini 2.5 Flash+Think†*	71.6	74.4	58.0	57.0	4.64	4.72
Qwen	QwQ-Plus	72.8	72.6	59.0	58.0	4.80	4.72
	Qwen-Plus	63.4	61.2	52.0	51.0	4.72	4.64
	Qwen-Max	30.6	30.4	26.0	25.0	2.24	2.24
	Qwen-Turbo	34.4	30.8	32.0	28.0	0.88	1.04
DeepSeek	DeepSeek-R1	84.2	**84.4**	74.0	**75.0**	3.76	3.20
	DeepSeek-V3.2	48.4	47.6	36.0	38.0	5.52	4.00

runs: green indicates a problem solved consistently on every run, red indicates consistent failure, and yellow indicates high trial-to-trial variability.

Figure 3 shows the distribution of per-problem pass rates aggregated across all models. Qwen-Turbo shows the most polarized profile (AV $=$ 0.0088), while Gemini 2.5 Flash has the highest AV (0.0704), indicating substantial trial-to-trial variability.

This view is useful because it exposes qualitatively different reliability profiles that similar aggregate pass rates can hide. A model with many $\hat{p}_i \approx 1$ and $\hat{p}_i \approx 0$ problems behaves differently from a model with many $\hat{p}_i \approx 0.4$ or 0.6 problems, even when their average run-level pass rate is similar.

We further note that retry-based recovery does not eliminate instability. PSR directly measures the fraction of tasks that do not require retries, while AV highlights how much of the remaining workload lies in a stochastic regime.

4.3 Difficulty Gradient and Failure Modes

To characterize where instability occurs, we performed a post-hoc aggregate analysis over the archived trial-level outputs from the completed paper runs. Pooling outcomes by benchmark difficulty reveals a pronounced gradient: run-level pass rates are highest on Easy problems (90.6%), lower on Medium problems (62.2%), and substantially lower on Hard problems (34.0%). This indicates that the overall accuracy–stability pattern is not driven by a small number of outlier tasks; it coexists with a broad increase in failure probability as algorithmic difficulty rises.

Failure modes are also non-uniform. Across the archived outputs, the dominant non-accept verdict

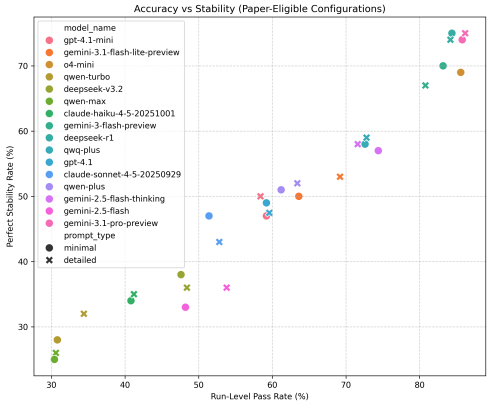

Figure 1: RLPR vs. PSR scatter plot across all 32 configurations (16 models $\times$ 2 prompt types). Points are colored by model family and shaped by prompt type. The diagonal marks PSR $=$ RLPR; points below indicate configurations where mean success overstates stable problem coverage.

is *Wrong Answer* (approximately 58% of all non-accept verdicts), followed by *Time Limit Exceeded* (approximately 27%) and *Runtime Error* (approximately 11%), with the remainder split between compilation errors and other platform-specific rejections. These proportions are approximate, derived from aggregate counts across all models and problems in the archived evaluation data, and should be interpreted as indicative of the overall distribution rather than precise per-model breakdowns. This ordering suggests that most failures arise from selecting an incorrect or incomplete algorithm rather than from pure formatting noise alone. At the same time, the prevalence of timeouts and runtime failures increases noticeably on Medium and Hard problems, indicating that instability is partly tied to implementation robustness and complexity-sensitive performance rather than only to binary conceptual success or failure.

These observations refine the interpretation of PSR and AV. A low PSR can reflect at least two qualitatively different regimes: algorithmic inconsistency, where runs alternate between accepted and wrong solutions, and implementation fragility, where some runs fail because the produced program is incomplete, inefficient, or execution-unsafe. Distinguishing these regimes is useful for deployment, because they imply different mitigation strategies such as prompt refinement, stricter output constraints, or downstream verification.

4.4 Reasoning vs. Standard Models

Reasoning-specialized models generate an internal chain-of-thought before producing a final answer, which could plausibly improve output consistency by structuring the solution process. We test whether this architectural distinction manifests as a systematically smaller accuracy–stability gap after controlling for overall accuracy level.

Grouping. We treat four models as reasoning-specialized (marked † in Table 1): DeepSeek-R1, QwQ-Plus, o4-mini, and Gemini 2.5 Flash+Think. The remaining 12 systems are classified as standard models. All conclusions from this analysis remain exploratory given the modest group sizes.

RLPR-controlled gap analysis. Because $\Delta = \mathrm{RLPR} - \mathrm{PSR}$ is itself correlated with RLPR (higher-accuracy models tend to have larger absolute gaps, as shown in Section 4.6), a raw comparison of mean gaps between groups would be confounded by accuracy level. To address this, we fit a linear model $\Delta \sim \mathrm{RLPR}$ across all 32 configurations ($R^2 = 0.40$, $p < 0.001$) and examine the residuals: a negative residual indicates a smaller gap than expected at the model's accuracy level, i.e., greater stability relative to accuracy.

Figure 4 shows the result. Reasoning models show strongly heterogeneous behavior: DeepSeek-R1 falls well below the regression line (mean residual ≈ -4.4), suggesting markedly better stability than expected for its accuracy level; o4-mini is near the line (residual ≈ -0.5); while QwQ-Plus (residual $\approx +1.7$) and Gemini 2.5 Flash+Think (residual $\approx +3.0$) both fall above it, showing larger-than-expected gaps. Across all eight reasoning-model configurations, the mean residual is -0.04, compared with $+0.01$ for the 24 standard-model configurations—a difference of 0.05 percentage points. A permutation test (10,000 iterations, two-sided) yields $p = 0.97$, providing no evidence that the residual difference is systematic.

Variance comparison. Average variance (AV) provides a complementary view of trial-to-trial variability. The four reasoning models have a mean AV of 4.30, compared with 3.81 for the 12 standard models—again showing no systematic advantage for reasoning-specialized architectures on this metric.

Interpretation. The four reasoning models in our sample do not exhibit systematically smaller

Figure 2: Cross-model stability heatmap (DETAILED prompt). Rows: 16 models sorted by RLPR descending. Columns: 100 problems grouped by difficulty tier with dashed separators, and ordered within each tier by decreasing mean pass rate. Cell color encodes empirical pass rate over $R = 5$ runs (green $= 1.0$, red $= 0.0$, yellow ≈ 0.5).

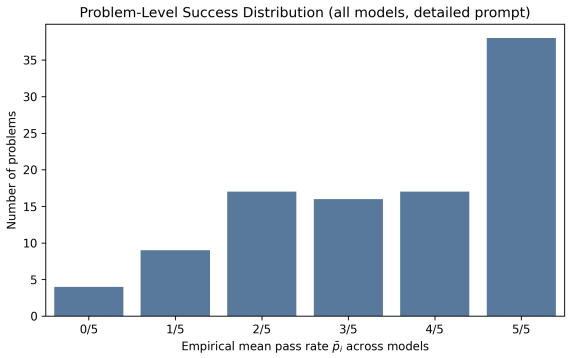

Figure 3: Problem-level success distribution (DETAILED prompt). The figure contains one panel per model (16 panels total), arranged in decreasing order of PSR. Each panel shows the fraction of that model's 100 problems falling in each empirical pass-probability bin $\hat{p}_i \in \{0/5, 1/5, \ldots, 5/5\}$, where $0/5$ means the model failed all 5 runs and $5/5$ means it passed all 5.

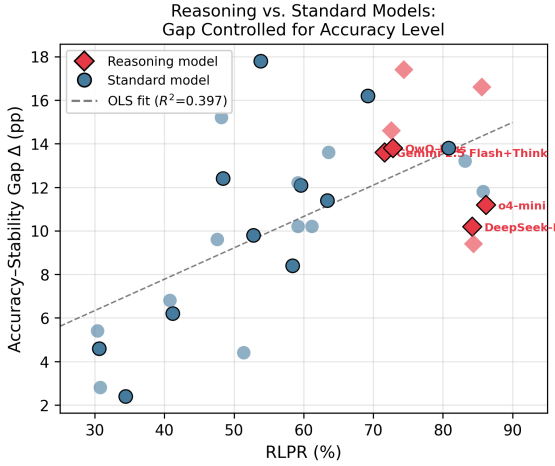

Figure 4: Accuracy–stability gap (Δ) vs. RLPR for all 32 configurations. Reasoning models (diamonds) and standard models (circles) are shown against an OLS fit. Points below the line have smaller gaps than expected at their accuracy level.

accuracy–stability gaps or lower trial-to-trial variability once accuracy level is accounted for. DeepSeek-R1 shows suggestive evidence of above-expected stability (residual ≈ -4.4 pp), a difference that is practically meaningful in deployment terms—a model producing 4 fewer gap percentage points than expected at its accuracy level offers substantially more retry-free coverage. However, with only four reasoning models in the sample, *the analysis is severely underpowered*: the permutation test ($p = 0.97$) should be read as "insufficient evidence from a small and heterogeneous sample" rather than as "the effect is negligible." A group comparison across $n = 4$ reasoning models has no meaningful power to detect a between-group effect of plausible magnitude, and the null result cannot be interpreted as evidence that reasoning architectures do not affect stability. DeepSeek-R1's individual pattern warrants verification in a larger and more diverse set of reasoning-specialized systems. The other three reasoning models do not show the same pattern, and the overall group difference remains negligible ($\Delta_{\text{resid}} = 0.05$ pp). The pattern is instead highly model-specific: chain-of-thought reasoning neither reliably improves nor worsens stability relative to accuracy within this small sample. The factors that drive trial-to-trial variability—prompt sensitivity, problem difficulty, and implementation fragility (Section 4.3)—appear to operate largely independently of whether a model employs explicit reasoning steps.

One important caveat applies to the Gemini 2.5 Flash vs. Gemini 2.5 Flash+Think comparison specifically. The two configurations differ not only in whether chain-of-thought reasoning is enabled, but also in output token budget: the standard variant used 4,096 tokens while the thinking variant used 24,576 tokens (including an 8,192-token thinking budget). The 17.8-point RLPR improvement from standard to thinking mode (53.8% $\rightarrow$ 71.6% under DETAILED) is therefore confounded between the

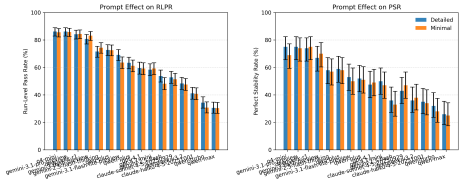

Figure 5: Prompt sensitivity: paired scatter of PSR under DETAILED vs. MINIMAL prompting for all 16 models. Points above the diagonal indicate that detailed prompting improves stability. Points are colored by model family.

reasoning mechanism and the token budget—if the standard Gemini 2.5 Flash configuration suffered truncated outputs due to the tighter budget, its low RLPR and high AV may partly reflect a capacity constraint rather than a model capability difference. We cannot disentangle these two factors from the current data, and the Gemini 2.5 Flash pair should not be interpreted as a clean ablation of thinking mode alone.

Expanding the sample of reasoning-specialized models—and controlling output budgets across standard and reasoning variants—remains an important direction for future work.

4.5 Prompt Sensitivity

Figure 5 visualizes the prompt effect as a paired scatter plot: each of the 16 models appears as a single point with its MINIMAL-prompt PSR on the x-axis and DETAILED-prompt PSR on the y-axis. Points above the diagonal indicate that detailed prompting improves stability.

Prompt effects are model-dependent and non-uniform. Most models show small differences between prompt conditions (within 3–5 percentage points of PSR). However, Claude Sonnet 4.5 shows a reversal, with higher PSR under MINIMAL (47.0%) than DETAILED (43.0%) prompting, while Gemini 3 Flash shows the opposite pattern (67.0% D vs. 70.0% M). In this dataset, prompt sensitivity is therefore observable but heterogeneous, and no single template uniformly dominates across all model families—consistent with prior findings that prompt sensitivity is task- and model-specific rather than universal (Zhuo et al., 2024). Because this is a controlled low-contrast prompt comparison, we interpret these results as evidence about a narrow prompt variation under fixed task specifications, not as a general statement about prompt engineering.

To assess whether these prompt differences are statistically compelling at the problem level, we ran exact McNemar tests on the paired PSR indicators for each model. Under this test, no model reaches $p < 0.05$; the largest observed PSR shifts are 4 percentage points (Qwen-Turbo in favor of DETAILED, Claude Sonnet 4.5 in favor of MINIMAL), and both correspond to $p = 0.219$ under exact paired testing. Across the 16 fully evaluated models, 11 favor DETAILED and 5 favor MINIMAL, with a mean absolute PSR difference of approximately 2.1 percentage points. We therefore interpret the prompt results as evidence of model-dependent directional variation, but not as strong evidence that either template yields a consistent advantage at the current benchmark size.

Two important caveats limit interpretability here. First, the comparison is deliberately low-contrast: both templates share identical XML structure, the same output constraints, and the same code-template slots, differing only in instruction density. This structural similarity means the comparison is unlikely to be sensitive enough to detect small systematic effects even if they exist; the null result may therefore reflect insufficient contrast between conditions rather than a true absence of prompt effects. Second, with $N = 100$ binary outcomes per model, a 4-percentage-point shift at $p = 0.22$ corresponds to a test with less than 20% power to detect an effect of this size, meaning that meaningful prompt effects cannot be ruled out on the basis of the current sample alone.

4.6 Accuracy–Stability Gap

Although RLPR and PSR are strongly correlated overall ($r = 0.985$), the gap $\Delta = \text{RLPR} - \text{PSR}$ is practically significant and non-uniform across models. Across all 32 configurations, Δ ranges from 2.4% (Qwen-Turbo, DETAILED) to 17.8% (Gemini 2.5 Flash, DETAILED). The gap peaks in the middle of the performance spectrum, where models succeed frequently enough to generate trial-to-trial variability but not consistently enough to eliminate it. This non-linearity means that a single RLPR ranking cannot be used to infer the magnitude of the accuracy–stability gap: two models with similar RLPR can differ substantially in how much of their accuracy converts to retry-free reliability. PSR makes this distinction explicit.

The high correlation does not imply that RLPR and PSR always agree on model rankings. A concrete reversal occurs within the GPT-4.1 family: GPT-4.1 leads GPT-4.1-mini by 1.2 percent-

age points on RLPR under DETAILED prompting (59.6% vs. 58.4%), but *trails* by 2.5 points on PSR (47.5% vs. 50.0%). In other words, RLPR ranks GPT-4.1 above GPT-4.1-mini, while PSR ranks them in the opposite order. This reversal—driven by GPT-4.1's larger accuracy–stability gap (12.1 pp vs. 8.4 pp)—illustrates that at $r = 0.985$, PSR and RLPR are still distinct enough to change deployment-relevant decisions: a practitioner selecting on RLPR would choose GPT-4.1, but one selecting on retry-free reliability would choose GPT-4.1-mini. Across the 16 evaluated models, at least one such ranking reversal is detectable under DETAILED prompting, confirming that PSR provides actionable ranking differences beyond what RLPR captures.

5 Scope and Practical Implications

This study is intentionally scoped to repeated-run correctness under a fixed evaluation harness. Stability remains practically important because deterministic programming tasks are used in settings that assume reproducibility. When a system is invoked repeatedly under the same input, instability creates concrete risks: non-reproducibility complicates debugging; workflow overhead arises from retry requirements; and validation burden increases from inconsistent outputs.

From a decision-theoretic perspective, selecting a model based solely on pass rate optimizes expected success but ignores the cost of stochastic failures. PSR provides a direct estimate of the workload fraction that is "retry-free." Even when two models have similar RLPR, they may differ substantially in PSR: this gap represents problems that require repeated invocations to reliably succeed in production, incurring latency, cost, and engineering complexity. A model ranked lower by RLPR could be preferable in deployment if its PSR is higher—a tradeoff invisible to single-run benchmarking. A concrete illustration from our results (Section 4.6): GPT-4.1 leads GPT-4.1-mini by 1.2 percentage points on RLPR but *trails* by 2.5 points on PSR, meaning a practitioner optimizing for retry-free reliability would make the opposite model selection compared to one optimizing for mean invocation success.

For this reason, we view repeated-run metrics as complementary rather than replacement metrics. RLPR remains useful for characterizing average invocation success, while PSR and AV summarize whether that success is concentrated in consistently solved problems or dispersed across unstable outcomes.

At the same time, our deployment interpretation remains intentionally narrow. In practical systems, generated code may flow through larger agentic or tool-mediated runtimes, introducing additional operational and security risks beyond single-turn correctness alone (as noted in concurrent work on agentic AI attack surfaces (Jiang et al., 2026)). Our evaluation does not attempt to measure those broader runtime-supply-chain concerns; instead, it isolates one lower-level component of reliability: whether the model repeatedly produces correct solutions for the same deterministic task.

6 Conclusion

We presented a repeated-run evaluation framework for deterministic programming tasks and used it to study the relationship between correctness and stability. Across 16 models, 100 problems, and 5 repeated runs per configuration (16,000 total instances), run-level pass rate and retry-free coverage are strongly correlated, but run-level accuracy consistently exceeds perfect stability. The resulting accuracy–stability gap is non-uniform and reaches up to 17.8 percentage points, peaking in the mid-accuracy tier where per-problem success probabilities cluster near 0.5. Importantly, PSR and RLPR can reverse model rankings even at high correlation ($r = 0.985$): GPT-4.1 leads GPT-4.1-mini on RLPR but trails on PSR, demonstrating that the two metrics capture deployment-relevant distinctions invisible to single-run reporting. Prompt effects and the reasoning-vs-standard architectural distinction are both heterogeneous and exploratory at the current sample size, suggesting that neither is a reliable lever for improving stability without model-specific tuning.

Future work should extend this framework in two directions: (1) replicating the repeated-run analysis on at least one non-coding deterministic NLP task (e.g., text-to-SQL) to validate generalizability, and (2) controlling output token budgets across standard and reasoning-mode variants to cleanly isolate the effect of chain-of-thought reasoning on stability. Taken together, these findings suggest that repeated-run evaluation is a necessary complement to standard accuracy reporting for any deterministic text-conditioned generation task in reproducibility-sensitive workflows.

Limitations

Several limitations qualify our results. First, the benchmark is relatively small ($N = 100$) and restricted to recent LeetCode-style problems. This design helps reduce direct overlap with heavily circulated historical benchmarks, but it does not eliminate contamination risk and should not be interpreted as proof of benchmark novelty. In particular, recently published contest problems are frequently discussed online within days of their release, meaning that recency alone is not sufficient to guarantee that evaluated models have not been exposed to equivalent problem statements through post-publication web text. The benchmark size is also a practical tradeoff: with 16 models, 2 prompt conditions, and 5 repeated runs per problem, even a 100-problem setup yields 16,000 judged evaluation instances. This scale is large enough to expose broad correctness–stability patterns, but still limited for detecting small prompt effects, which is consistent with the non-significant prompt comparisons reported in Section 4.

Second, our repeated-run protocol is intentionally narrow: we vary prompt template and repeated sampling while holding the evaluation harness fixed. As a result, the study does not characterize the effects of broader prompt engineering, tool use, self-repair, retrieval, test-time selection, or multi-turn interaction.

Third, model access is mediated through provider APIs and platform infrastructure. Even though task grading is deterministic, we do not control all backend implementation details, model refreshes, or service-side changes that may affect outputs over time. Our conclusions should therefore be interpreted relative to documented model snapshots and access dates rather than as timeless statements about any provider family.

Fourth, our summary metrics compress distinct failure modes into a binary pass/fail outcome. This is appropriate for end-task correctness, but it does not by itself explain whether instability arises from reasoning errors, formatting mistakes, runtime failures, or judge-specific edge cases.

Fifth, a specific methodological caveat applies to o4-mini. The o-series API does not expose temperature or top-p controls; the model's internal decoding regime is not documented and may differ substantially from the $T = 0.3$ setting used for all other models. If o4-mini's effective temperature is near zero—producing near-deterministic outputs across runs—then its measured PSR and AV reflect a qualitatively different quantity from the other systems: we would be observing the near-absence of stochasticity rather than meaningful stability under repeated stochastic sampling. This limits direct comparability of o4-mini's repeated-run metrics with those of other models, and any high PSR for o4-mini should be interpreted cautiously as a potential artifact of reduced output variance rather than as evidence of robust problem-solving reliability.

Finally, our conclusions are limited to Python code generation on deterministic coding tasks with executable tests. They should not be read as a complete account of reliability for other programming languages, for open-ended generation tasks, or for production systems that include additional orchestration and verification layers.

Ethics Statement

This paper evaluates commercial and open-access LLM APIs on deterministic programming tasks. The study does not involve human subjects, personal data collection, or annotation labor. The main ethical considerations are instead tied to platform use and deployment interpretation. First, the evaluation relies on automated submission to an external online judge. Such use should be understood as research benchmarking under the platform's operational constraints, and any public release should avoid exposing credentials, session artifacts, or procedures that would enable abusive automation. Second, the evaluated models are accessed through third-party APIs whose outputs and availability may be governed by provider-specific terms of service and may change over time. Our reported results should therefore be interpreted as measurements of accessed systems under a documented evaluation protocol, not as immutable properties of any provider's model family.

More broadly, stronger performance on coding benchmarks can increase both beneficial and harmful capabilities. Better repeated-run reliability can support legitimate software engineering workflows, but it may also lower the barrier to generating code in settings where security review is weak. For this reason, we frame our contribution as an evaluation methodology rather than as an endorsement of fully autonomous code deployment.

References

Pranjal Aggarwal, Aman Madaan, Yiming Yang, and Mausam. 2023. Let's sample step by step: Adaptive-consistency for efficient reasoning and coding with LLMs. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, pages 12375–12396.

Miguel Angel Alvarado Gonzalez and 1 others. 2025. Do repetitions matter? Strengthening reliability in LLM evaluations. *arXiv preprint arXiv:2509.24086*. Preprint; arXiv:2509.24086.

Berk Atil, Sarp Aykent, Harun Tuncer, and 1 others. 2024. LLM stability: A detailed analysis with some surprises. *arXiv preprint arXiv:2408.04667*.

Jacob Austin, Augustus Odena, Maxwell Nye, Maarten Bosma, Henryk Michalewski, David Dohan, Ellen Jiang, Carrie Cai, Michael Terry, Quoc Le, and 1 others. 2021. Program synthesis with large language models. *arXiv preprint arXiv:2108.07732*.

Bradley Brown, Jordan Juravsky, Ryan Ehrlich, Ronald Clark, Quoc V. Le, Christopher Ré, and Azalia Mirhoseini. 2024. Large language monkeys: Scaling inference compute with repeated sampling. *arXiv preprint arXiv:2407.21787*.

Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Ponde de Oliveira Pinto, Jared Kaplan, Harri Edwards, Yuri Burda, Nicholas Joseph, Greg Brockman, and 1 others. 2021. Evaluating large language models trained on code. *arXiv preprint arXiv:2107.03374*.

Chunyuan Deng, Yilun Zhao, Xiangru Tang, Mark Gerstein, and Arman Cohan. 2023. Investigating data contamination in modern benchmarks for large language models. *arXiv preprint arXiv:2311.09783*.

Xueying Du, Mingwei Liu, Kaixin Wang, Hanlin Wang, Junwei Liu, Yixuan Chen, Junjie Feng, Guangyu Zhu, Shang-Wei Li, and Yang Liu. 2024. Evaluating large language models in class-level code generation. *arXiv preprint arXiv:2308.01861*.

Dawei Gao, Haibin Wang, Yaliang Li, Xiuyu Sun, Yichen Qian, Bolin Ding, and Jingren Zhou. 2024. Text-to-SQL empowered by large language models: A benchmark evaluation. *Proceedings of the VLDB Endowment*, 17(5):1132–1145.

Chuan Guo, Geoff Pleiss, Yu Sun, and Kilian Q Weinberger. 2017. On calibration of modern neural networks. In *Proceedings of the 34th International Conference on Machine Learning*, pages 1321–1330.

Dan Hendrycks, Steven Basart, Saurav Kadavath, Mantas Mazeika, Andy Zou, Dawn Song, and Jacob Steinhardt. 2021. Measuring coding challenge competence with APPS. *arXiv preprint arXiv:2105.09938*.

Naman Jain, King Han, Alex Gu, Wen-Ding Li, Fanjia Yan, Tianjun Zhang, Sida Wang, Armando Solar-Lezama, Koushik Sen, and Ion Stoica. 2024. Livecodebench: Holistic and contamination free evaluation of large language models for code. *arXiv preprint arXiv:2403.07974*.

Xiaochong Jiang, Shiqi Yang, Wenting Yang, Yichen Liu, and Cheng Ji. 2026. Agentic ai as a cybersecurity attack surface: Threats, exploits, and defenses in runtime supply chains. *Preprint*, arXiv:2602.19555.

Carlos E. Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik R. Narasimhan. 2024. SWE-bench: Can language models resolve real-world github issues? *arXiv preprint arXiv:2310.06770*.

Takeshi Kojima, Shixiang Shane Gu, Machel Reid, Yutaka Matsuo, and Yutaka Iwasawa. 2022. Large language models are zero-shot reasoners. *arXiv preprint arXiv:2205.11916*.

Yujia Li, David Choi, Junyoung Chung, Nate Kushman, Julian Schrittwieser, Rémi Leblond, Tom Eccles, James Keeling, Felix Gimeno, Agustin Dal Lago, and 1 others. 2022. Competition-level code generation with AlphaCode. *arXiv preprint arXiv:2203.07814*.

Jiawei Liu, Chunqiu Steven Xia, Yuyao Wang, and Lingming Zhang. 2024. Is your code generated by chatgpt really correct? rigorous evaluation of large language models for code generation. *Advances in Neural Information Processing Systems*, 36.

Xuezhi Wang, Jason Wei, Dale Schuurmans, Quoc Le, Ed H. Chi, Sharan Narang, Aakanksha Chowdhery, and Denny Zhou. 2022. Self-consistency improves chain of thought reasoning in language models. *arXiv preprint arXiv:2203.11171*.

Yike Zhang, Zuodong Xiang, and Hailu Xu. 2026. Performance-efficiency trade-offs in human preference prediction: A comparative study of traditional machine learning and large language models. In *Proceedings of the 31st IEEE Symposium on Computers and Communications (ISCC)*. IEEE. To appear.

Jingming Zhuo, Songyang Zhang, Xinyu Fang, Haodong Duan, Dahua Lin, and Kai Chen. 2024. ProSA: Assessing and understanding the prompt sensitivity of LLMs. *arXiv preprint arXiv:2410.12405*.

A Prompt Templates

A.1 Detailed Prompt

```
<role>
You are an expert Python algorithm engineer
    solving deterministic
LeetCode-style problems.
</role>

<task>
Produce a correct Python solution that fits
    the provided template and
```

```
is optimized for the problem constraints.
</task>

<instructions>
1. Read the full problem statement,
    constraints, and template before
   writing code.
2. Produce the best practical algorithm for
    the stated constraints,
   aiming for the optimal time complexity
    when possible.
3. Do not return a brute-force, quadratic,
    exponential, or placeholder
   solution when the constraints require a
    more efficient algorithm.
4. Use the required method signature from
    the template exactly.
5. Ensure the algorithm is complete and
    submission-ready, not a sketch
   or partial attempt.
6. Handle edge cases implied by the
    statement and constraints.
7. Return one complete Python solution that
    can be submitted directly.
</instructions>

<format_requirements>
Return plain Python source code only.
Start directly with the code.
Do not include explanations, markdown fences,
     or surrounding commentary.
The code must be valid, runnable Python 3
    with no syntax errors.
The code must fit the provided code template
     without requiring manual edits.
The final code should be written for
    Accepted-style performance, not just
sample-case correctness.
Only include comments when they clarify non-
    obvious logic.
</format_requirements>

<problem>
{problem_description}
</problem>

<code_template>
{code_template}
</code_template>
```

A.2 Minimal Prompt

```
<role>
You are a Python coding assistant solving a
    deterministic
LeetCode-style problem.
</role>

<task>
Write one correct and efficient Python
    solution that fits the provided
template.
</task>

<instructions>
1. Infer an algorithm that matches the
    constraints before writing code.
2. Prefer the optimal or near-optimal
    solution for the input limits.
3. Do not output brute-force code unless the
     constraints clearly make
   it acceptable.
4. Use the exact method signature from the
    template.
5. Return only final submission code.
</instructions>

<format_requirements>
Return plain Python source code only.
Start directly with the code.
Do not include explanations, markdown fences,
     or extra text.
The code must be valid, runnable Python 3
    with no syntax errors.
The code must fit the provided code template
     without requiring manual edits.
The solution must be efficient enough to
    avoid obvious Time Limit
Exceeded outcomes under the given
    constraints.
</format_requirements>

<problem>
{problem_description}
</problem>

<code_template>
{code_template}
</code_template>
```

B Model Snapshots

Table 3: Model snapshots used in the experiments. The API identifier column records the concrete model string used by the evaluation pipeline. Access dates are derived from the archived run dates for the corresponding completed paper runs.

Display Name	Provider	API Identifier	Access Date
GPT-4.1	OpenAI	gpt-4.1	2026-04-01
GPT-4.1-mini	OpenAI	gpt-4.1-mini	2026-03-31
o4-mini	OpenAI	o4-mini	2026-04-06
Claude Sonnet 4.5	Anthropic	claude-sonnet-4-5-20250929	2026-04-03
Claude Haiku 4.5	Anthropic	claude-haiku-4-5-20251001	2026-04-03
Gemini 3.1 Pro (preview)	Google	gemini-3.1-pro-preview	2026-03-28
Gemini 3 Flash (preview)	Google	gemini-3-flash-preview	2026-03-29
Gemini 3.1 Flash-Lite (preview)	Google	gemini-3.1-flash-lite-preview	2026-03-30
Gemini 2.5 Flash	Google	gemini-2.5-flash	2026-04-05
Gemini 2.5 Flash+Think	Google	gemini-2.5-flash (thinkingBudget=8192)	2026-04-05
QwQ-Plus	Qwen	qwq-plus	2026-03-23
Qwen-Plus	Qwen	qwen-plus	2026-03-29
Qwen-Max	Qwen	qwen-max	2026-03-23
Qwen-Turbo	Qwen	qwen-turbo	2026-03-23
DeepSeek-R1	DeepSeek	deepseek-r1	2026-03-24
DeepSeek-V3.2	DeepSeek	deepseek-v3.2	2026-03-25